CSSE3113 – Introduction to Software Engineering

Software Project Scheduling

Eight Reasons for Late Software Delivery

- An unrealistic deadline established by someone outside the software engineering group and forced on managers and practitioners within the group
- Changing customer requirements that are not reflected in schedule changes
- An honest underestimate of the amount of effort and /or the number of resources that will be required to do the job
- Predictable and/or unpredictable risks that were not considered when the project commenced

Eight Reasons for Late Software Delivery

- Technical difficulties that could not have been foreseen in advance
- Human difficulties that could not have been foreseen in advance
- Miscommunication among project staff that results in delays
- A failure by project management to recognize that the project is falling behind schedule and a lack of action to correct the problem

Basic Principles for Project Scheduling

- **Compartmentalization**
  - The project must be compartmentalized into a number of manageable activities, actions, and tasks; both the product and the process are decomposed
- **Interdependency**
  - The interdependency of each compartmentalized activity, action, or task must be determined
  - Some tasks must occur in sequence while others can occur in parallel
  - Some actions or activities cannot commence until the work product produced by another is available
- **Time allocation**
  - Each task to be scheduled must be allocated some number of work units
  - In addition, each task must be assigned a start date and a completion date that are a function of the interdependencies
  - Start and stop dates are also established based on whether work will be conducted on a full-time or part-time basis

Basic Principles for Project Scheduling

- **Effort validation**
  - Every project has a defined number of people on the team
  - As time allocation occurs, the project manager must ensure that no more than the allocated number of people have been scheduled at any given time
- **Defined responsibilities**
  - Every task that is scheduled should be assigned to a specific team member
- **Defined outcomes**
  - Every task that is scheduled should have a defined outcome for software projects such as a work product or part of a work product
  - Work products are often combined in deliverables
- **Defined milestones**
  - Every task or group of tasks should be associated with a project milestone
  - A milestone is accomplished when one or more work products has been reviewed for quality and has been approved

Relationship Between People and Effort

- Common management myth:

If we fall behind schedule, we can always add more programmers and catch up later in the project

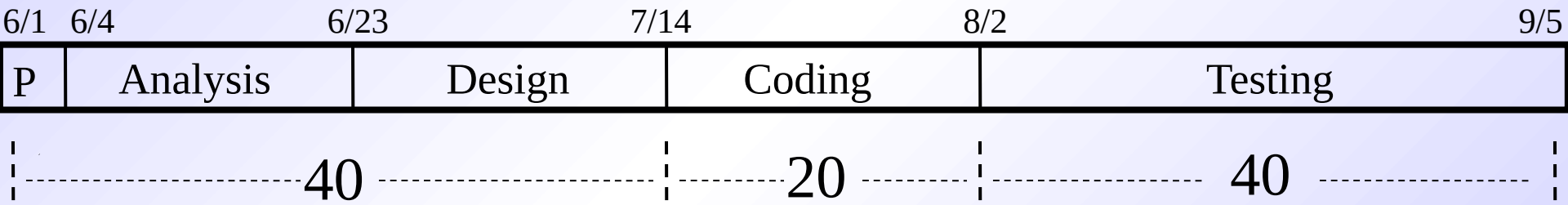
  - This practice actually has a disruptive effect and causes the schedule to slip even further
  - The added people must learn the system
  - The people who teach them are the same people who were earlier doing the work
  - During teaching, no work is being accomplished
  - Lines of communication (and the inherent delays) increase for each new person added

40-20-40 Distribution of Effort

- A recommended distribution of effort across the software process is 40% (analysis and design), 20% (coding), and 40% (testing)
- Work expended on project planning rarely accounts for more than 2 - 3% of the total effort
- Requirements analysis may comprise 10 - 25%
  - Effort spent on prototyping and project complexity may increase this
- Software design normally needs 20 – 25%
- Coding should need only 15 - 20% based on the effort applied to software design
- Testing and subsequent debugging can account for 30 - 40%
  - Safety or security-related software requires more time for testing

40-20-40 Distribution of Effort

Example: 100-day project

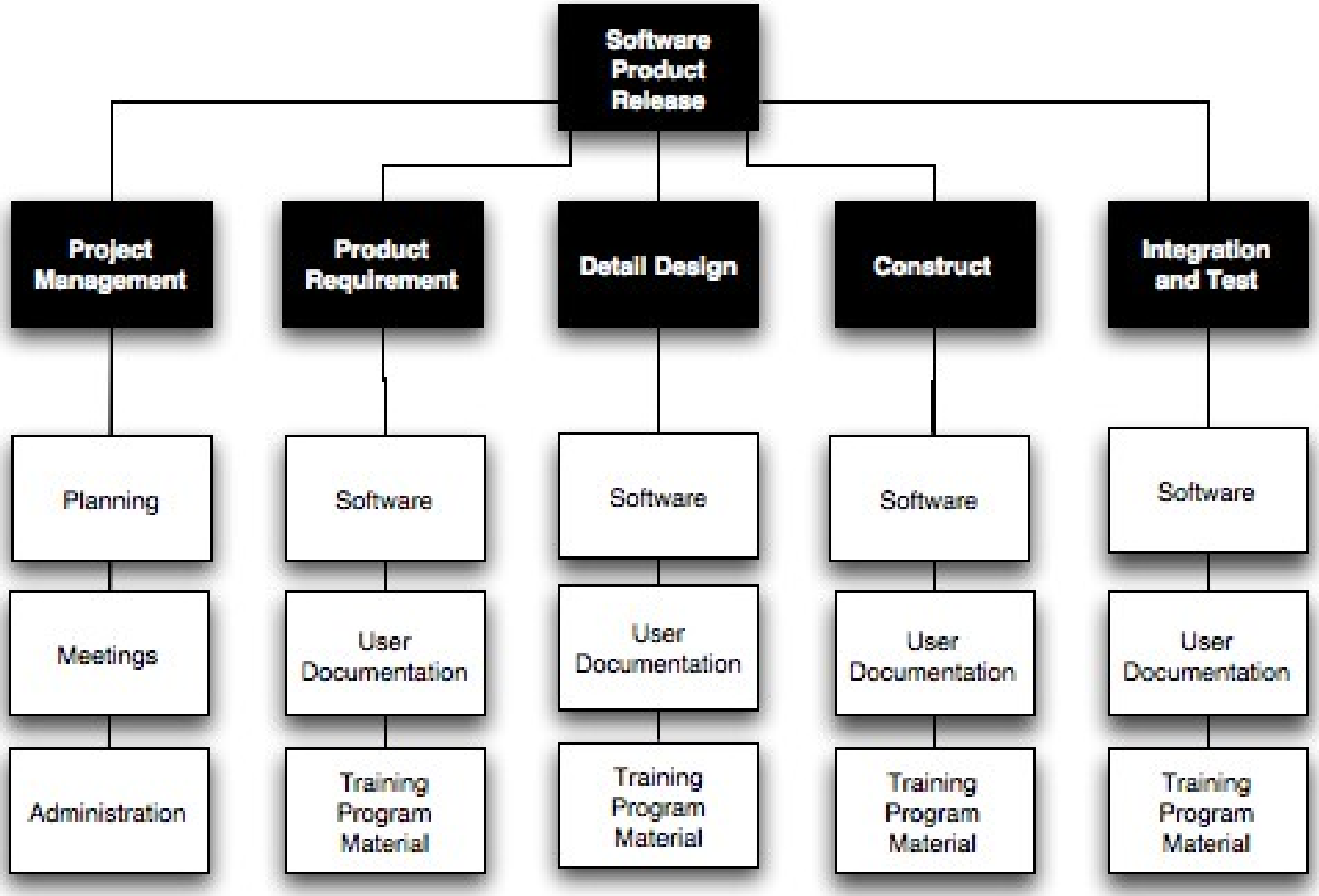


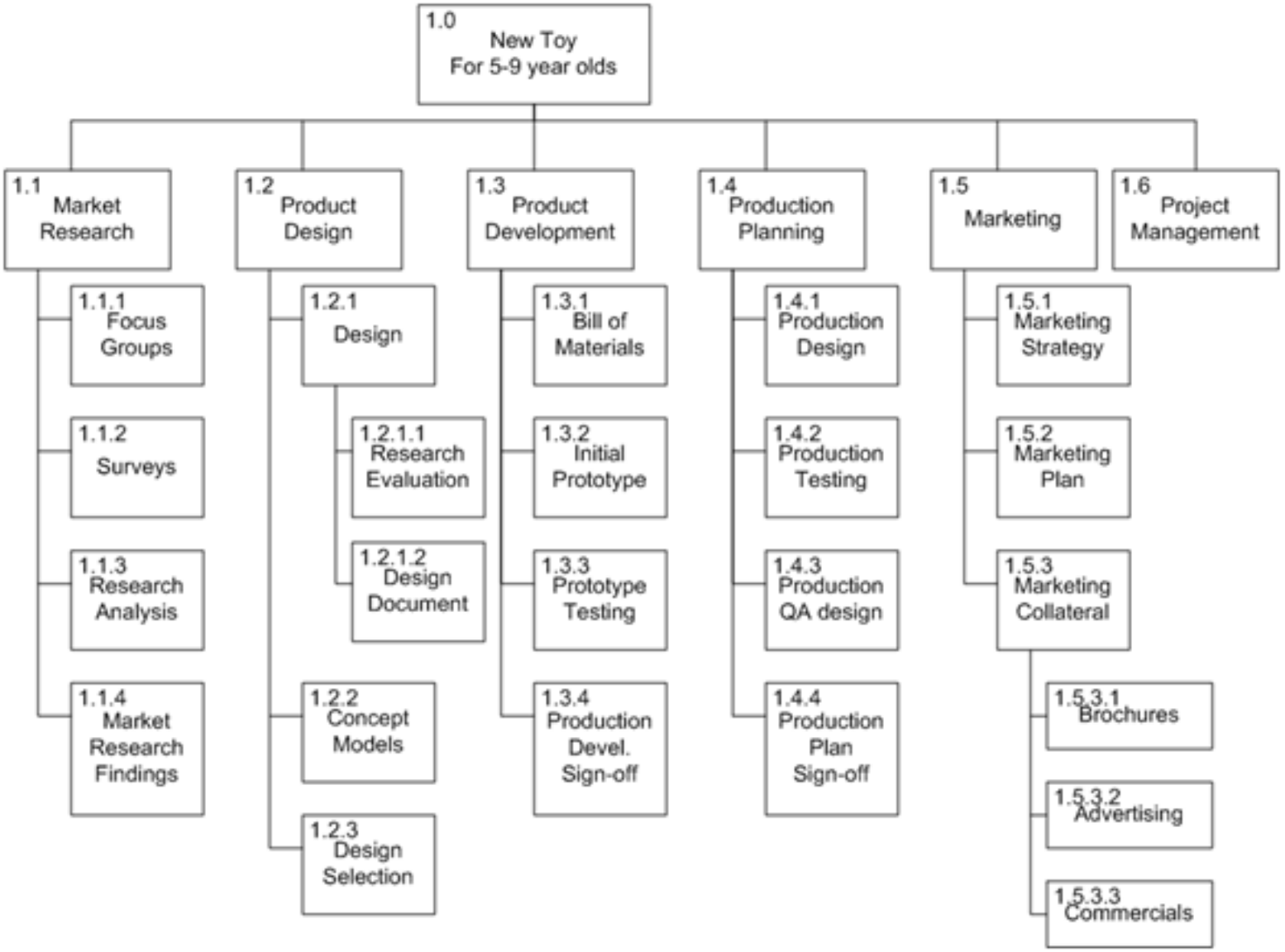
Task Network

Defining a Task Set

- A task set is the work breakdown structure for the project
- No single task set is appropriate for all projects and process models
  - It varies depending on the project type and the degree of rigor (based on influential factors) with which the team plans to work
- The task set should provide enough discipline to achieve high software quality
  - But it must not burden the project team with unnecessary work

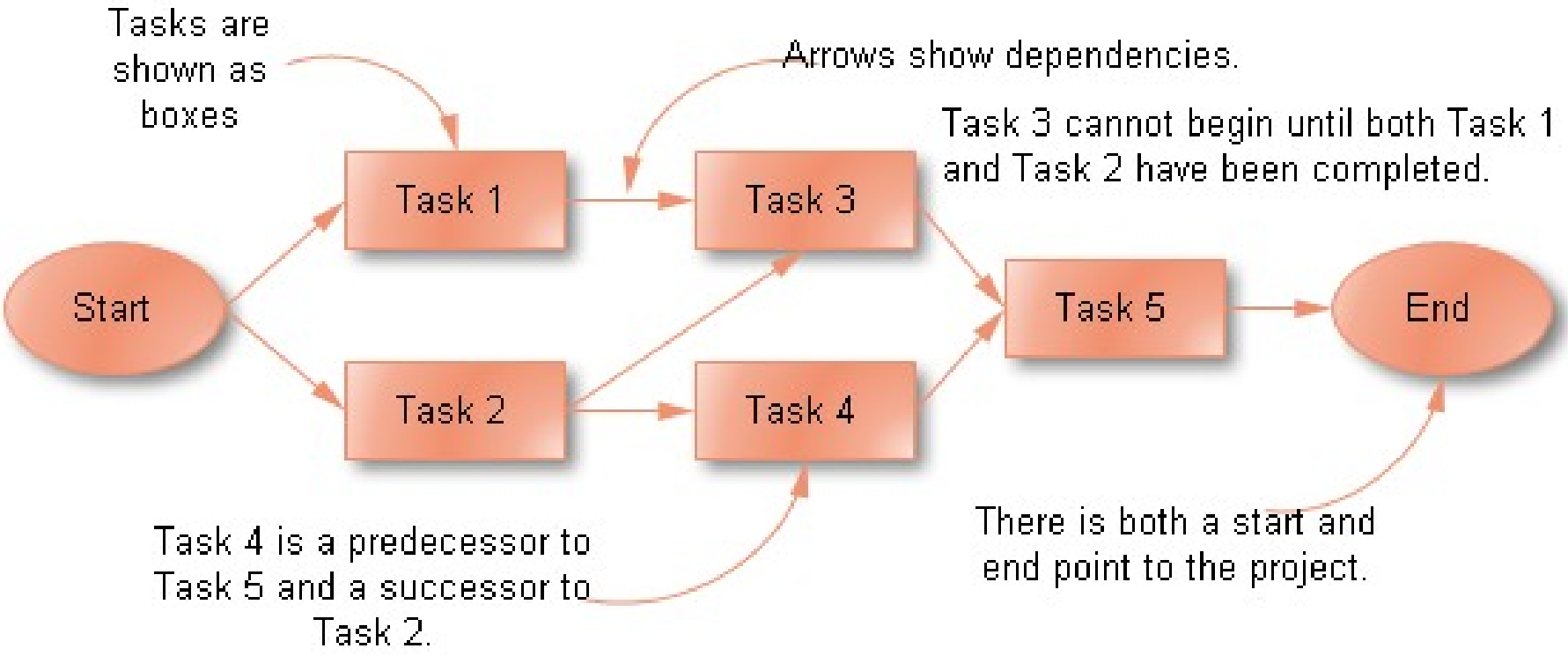
An example is given in next two slides.


Software Product Release
Project Management
Product Requirement
Detail Design
Construct
Integration and Test
Planning
Meetings
Administration
Software
User Documentation
Training Program Material
Software
User Documentation
Training Program Material
Software
User Documentation
Training Program Material
Software
User Documentation
Training Program Material

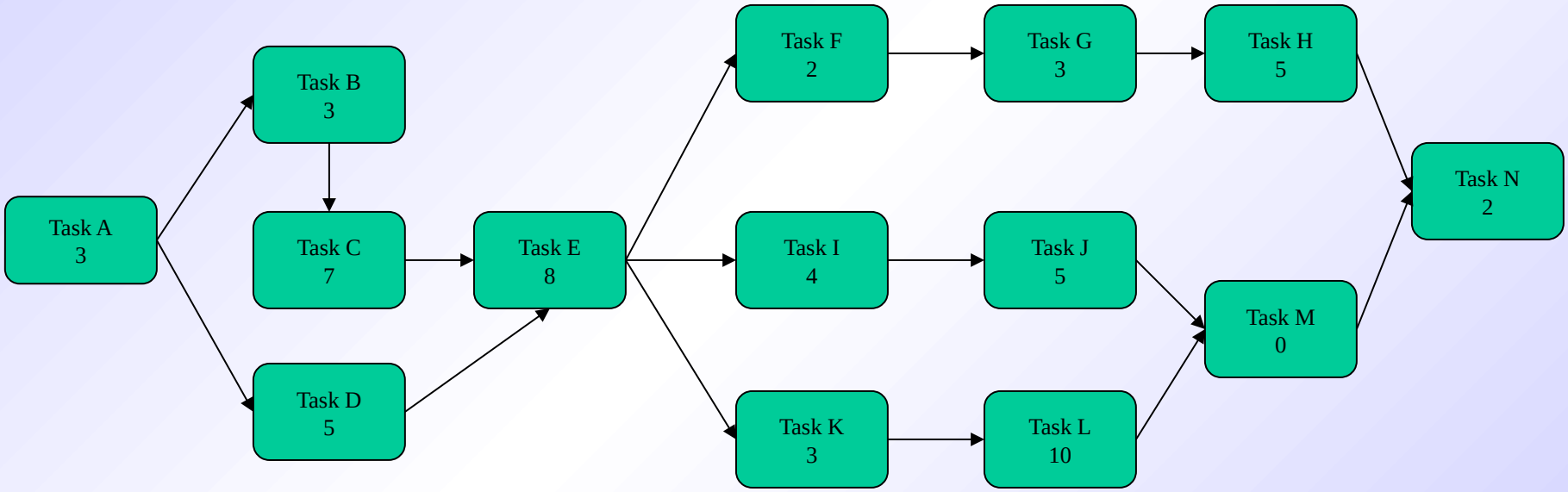

1.0
New Toy
For 5-9 year olds
1.1
Market Research
1.1.1
Focus Groups
1.1.2
Surveys
1.1.3
Research Analysis
1.1.4
Market Research Findings
1.2
Product Design
1.2.1
Design
1.2.1.1
Research Evaluation
1.2.1.2
Design Document
1.2.2
Concept Models
1.2.3
Design Selection
1.3
Product Development
1.3.1
Bill of Materials
1.3.2
Initial Prototype
1.3.3
Prototype Testing
1.3.4
Production Devel. Sign-off
1.4
Production Planning
1.4.1
Production Design
1.4.2
Production Testing
1.4.3
Production QA design
1.4.4
Production Plan Sign-off
1.5
Marketing
1.5.1
Marketing Strategy
1.5.2
Marketing Plan
1.5.3
Marketing Collateral
1.5.3.1
Brochures
1.5.3.2
Advertising
1.5.3.3
Commercials
1.6
Project Management

Purpose of a Task Network

- Also called an activity network
- It is a graphic representation of the task flow for a project
- It depicts task length, sequence, concurrency, and dependency
- Points out inter-task dependencies to help the manager ensure continuous progress toward project completion
- The **critical path**
  - A **single** path leading from **start to finish** in a task network
  - It contains the sequence of tasks that **must be completed on schedule** if the project as a whole is to be completed on schedule
  - It also determines the **minimum duration** of the project

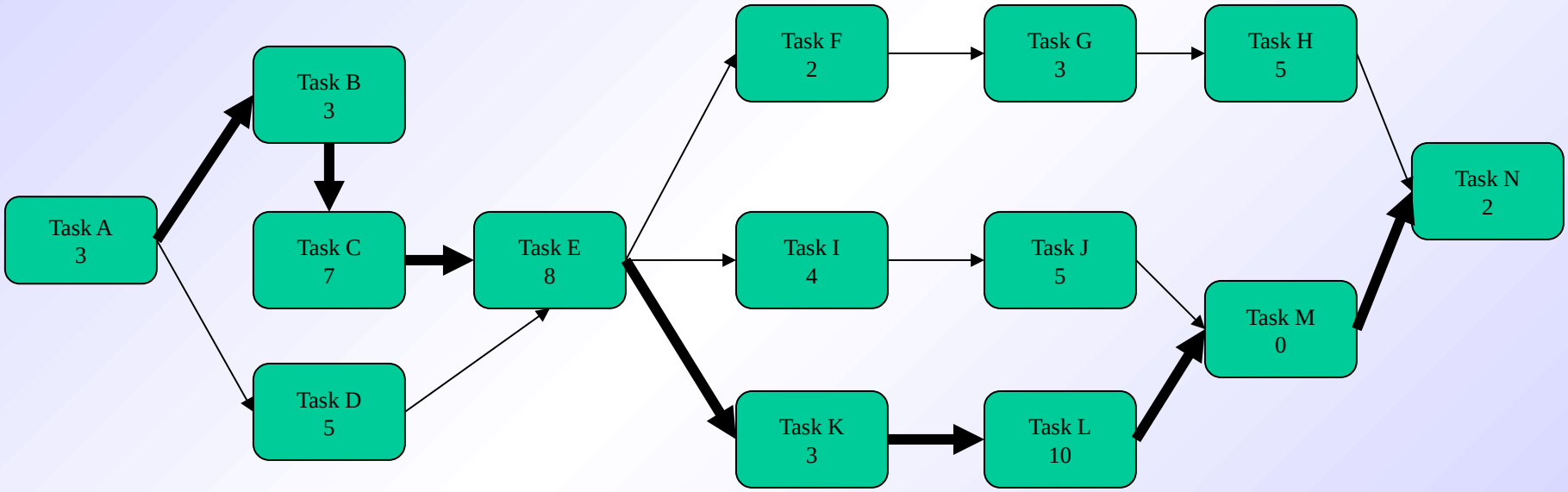

Tasks are shown as boxes
Arrows show dependencies.
Task 3 cannot begin until both Task 1 and Task 2 have been completed.
Start
Task 1
Task 2
Task 3
Task 4
Task 5
End
Task 4 is a predecessor to Task 5 and a successor to Task 2.
There is both a start and end point to the project.

Example Task Network



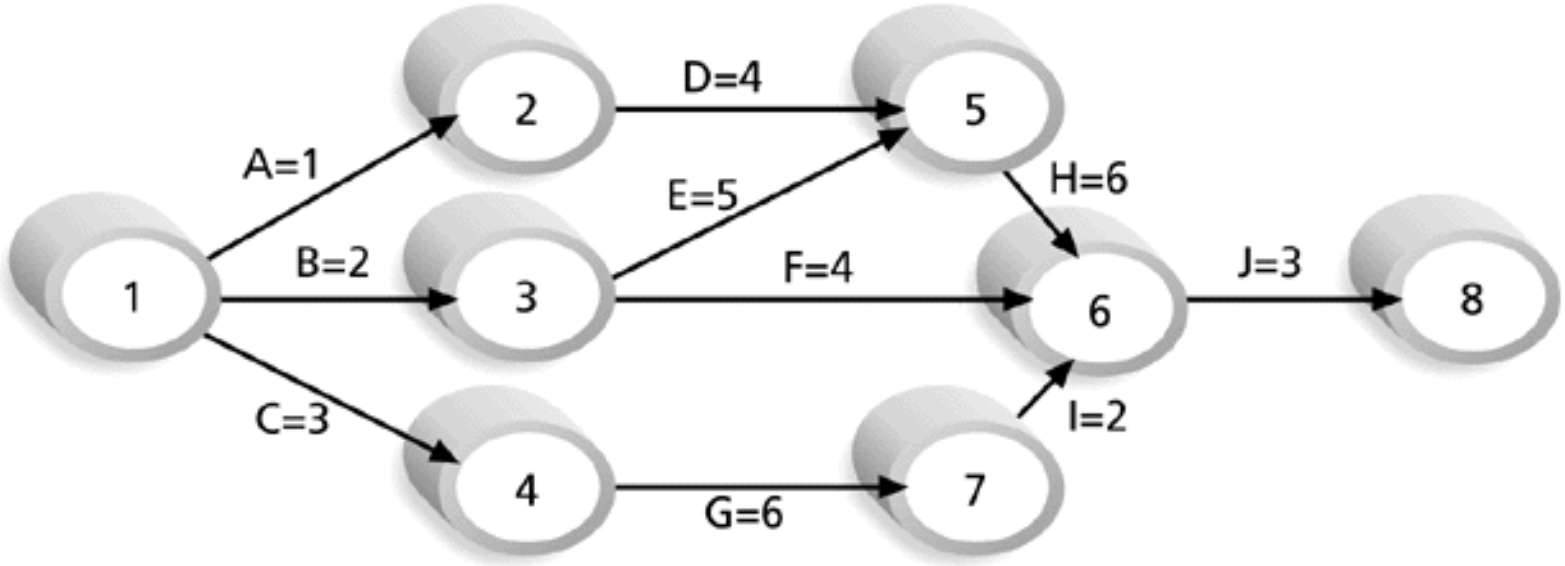
Where is the critical path and what tasks are on it?

Example Task Network with Critical Path Marked



Critical path: A-B-C-E-K-L-M-N

Determining the Critical Path for Project X



Note: Assume all durations are in days.

Path 1:	A-D-H-J	Length = 1+4+6+3 = 14 days
Path 2:	**B-E-H-J**	**Length = 2+5+6+3 = 16 days**
Path 3:	B-F-J	Length = 2+4+3 = 9 days
Path 4:	C-G-I-J	Length = 3+6+2+3 = 14 days

Since the critical path is the longest path through the network diagram, Path 2, B-E-H-J, is the critical path for Project X.

Timeline Chart

Mechanics of a Timeline Chart

- Also called a Gantt chart; invented by Henry Gantt, industrial engineer, 1917
- All project tasks are listed in the far left column
- The next few columns may list the following for each task:
  - projected start date
  - projected stop date
  - projected duration
  - actual start date
  - actual stop date
  - actual duration
  - task inter-dependencies (i.e., predecessors)

Mechanics of a Timeline Chart

- To the far right are columns representing dates on a calendar
- The length of a horizontal bar on the calendar indicates the duration of the task
- When multiple bars occur at the same time interval on the calendar, this implies task concurrency
- A diamond in the calendar area of a specific task indicates that the task is a milestone; a milestone has a time duration of zero